

# PROGRAMACIÓN I

Presencial **Turno noche**

Codificación

# Archivos .h y .cpp

06. Funciones

# ¿Para qué separar archivos?

Hasta ahora, nuestros proyectos eran soluciones a pequeños problemas. Pero, a medida que avanzamos, los ejercicios fueron requiriendo un mayor nivel de desacoplamiento, por lo que comenzamos a separar en **funciones**.

Sin embargo, cuando el programa crece y aparecen muchas funciones, y con distintas finalidades, mantener el control puede volverse complicado. Y sin un buen orden, trabajar con comodidad en proyectos más grandes se hace cada vez más difícil.

Dividir el programa en distintos archivos (**.h** y **.cpp**) nos permite organizar el trabajo.

```
Proyecto/  
|  
|—— source/  
|   |—— main.cpp  
|   |—— funciones.cpp  
|  
|—— headers/  
    |—— funciones.h
```

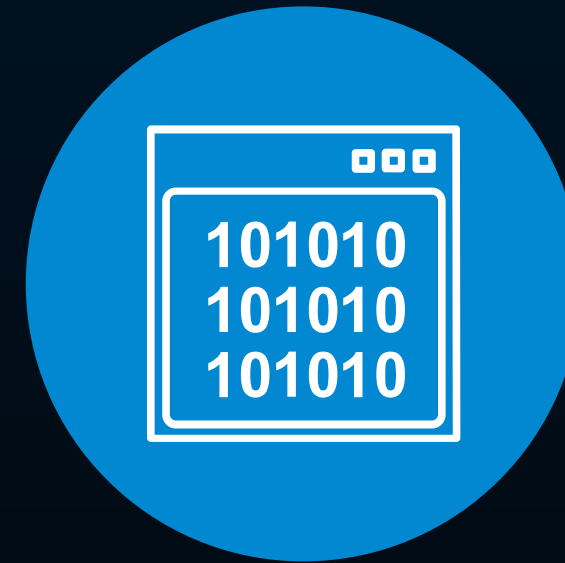
# ¿Qué es una archivo `.h` y `.cpp`?

Cuando un programa crece, no solo necesitamos más funciones, también necesitamos ordenarlas. Para eso aparecen **dos tipos de archivos**:



## Archivos `.cpp`

Son los archivos donde vamos a definir o **desarrollar las funciones**.



## Archivos `.h`

Son los archivos donde se **declaran las funciones** que existen.

# Hasta ahora

main.cpp

```
// PROTOTIPO
void mostrarMensaje();

int main()
{
    mostrarMensaje(); // LLAMADO

    return 0;
}

// DEFINICION
void mostrarMensaje()
{
    cout << "Hola desde la función" << endl;
}
```



# Hasta ahora

main.cpp

```
// PROTOTIPO
void mostrarMensaje();

int main()
{
    mostrarMensaje(); // LLAMADO

    return 0;
}
```

funciones.h

```
// DEFINICION
void mostrarMensaje()
{
    cout << "Hola desde la función" << endl;
}
```

funciones.cpp

# Desde ahora

```
// PROTOTIPO  
void mostrarMensaje();
```

funciones.h

```
int main()  
{  
    mostrarMensaje(); // LLAMADO  
  
    return 0;  
}
```

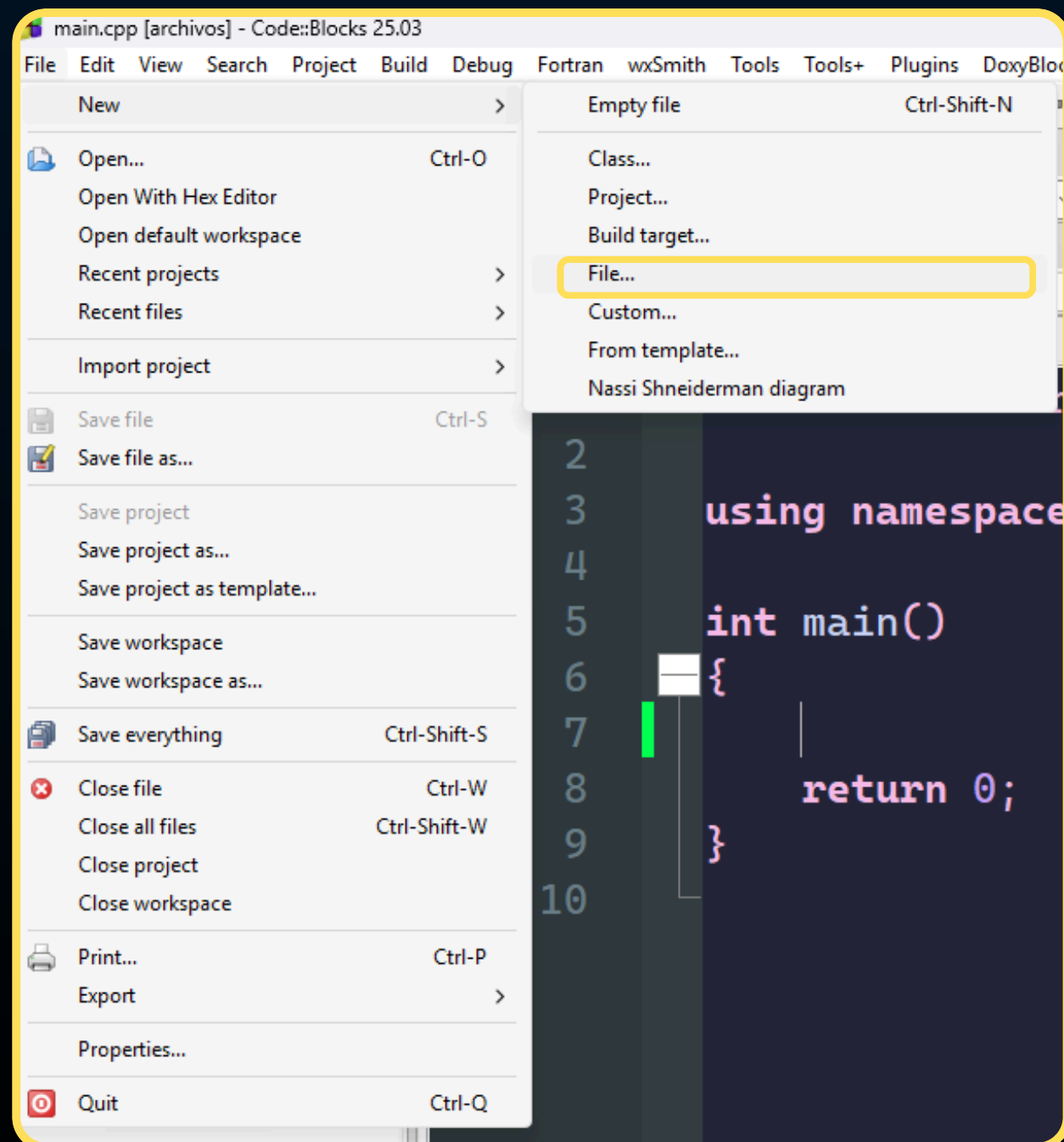
main.cpp

```
// DEFINICION  
void mostrarMensaje()  
{  
    cout << "Hola desde la función" << endl;  
}
```

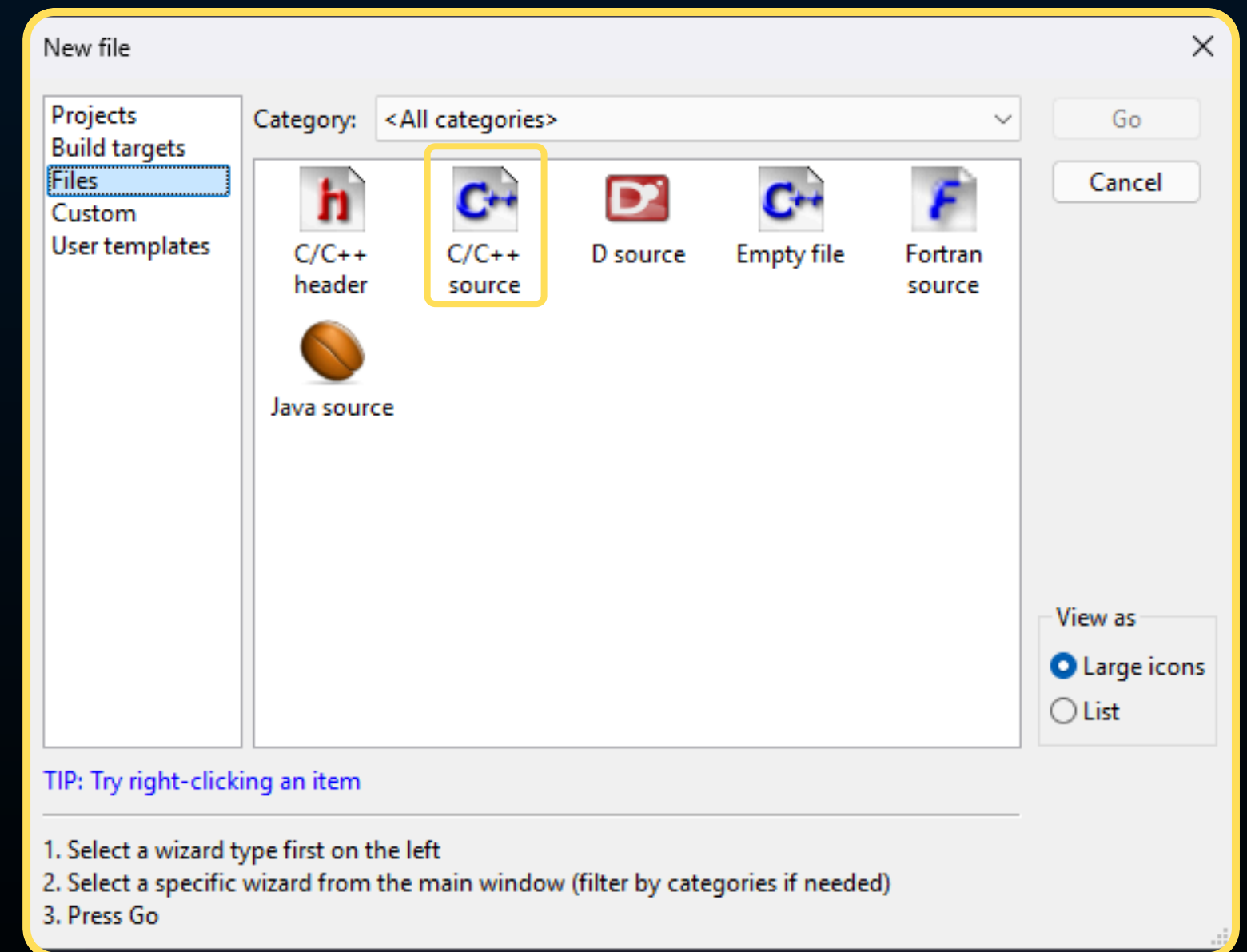
funciones.cpp

# ¿Cómo creamos archivos .cpp?

1 File > New > File ...



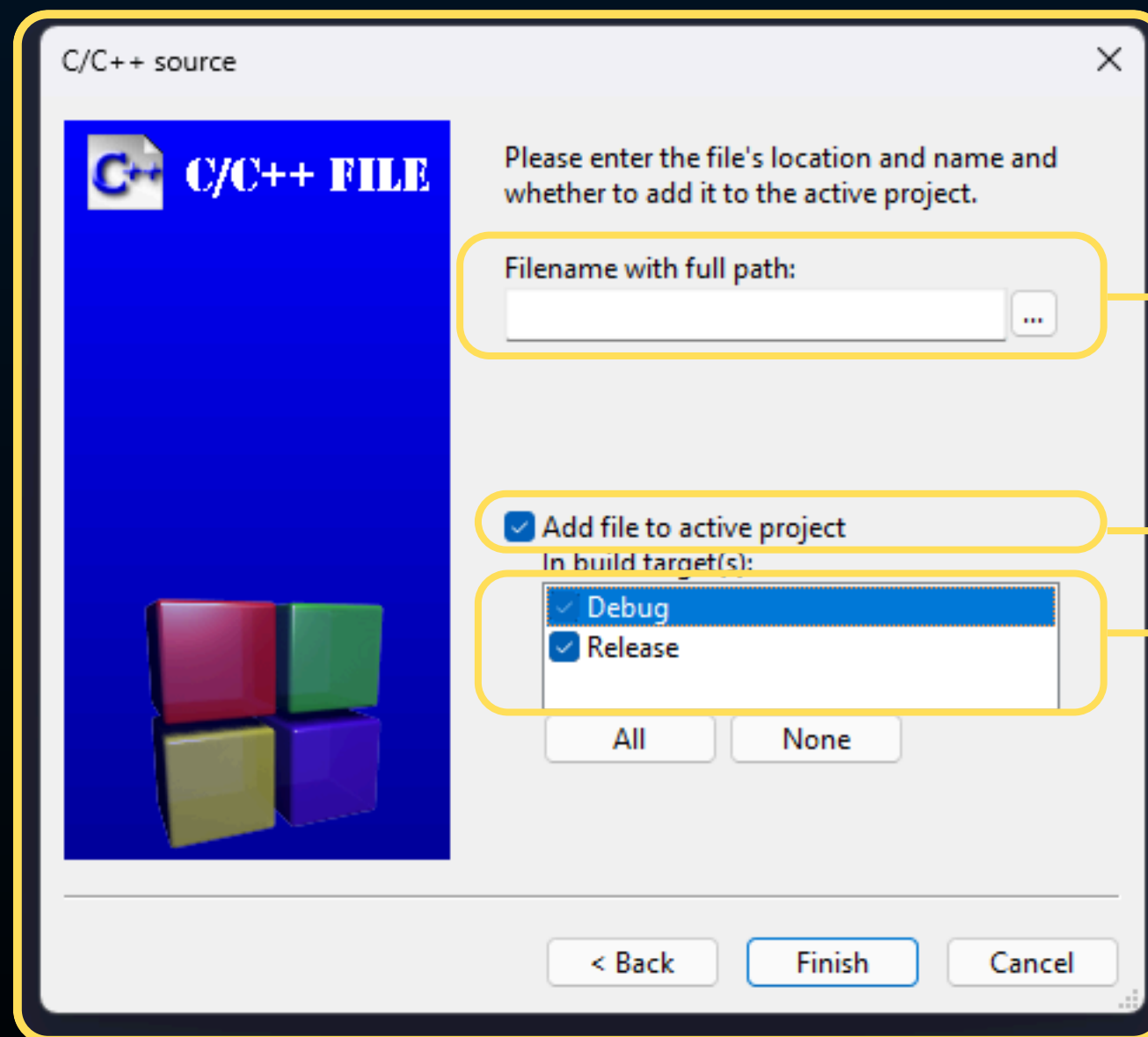
2 Files > C/C++ source





# ¿Cómo creamos archivos .cpp?

## 3 C/C++ source



### Directorio del archivo.

Mantenemos el mismo que el proyecto.

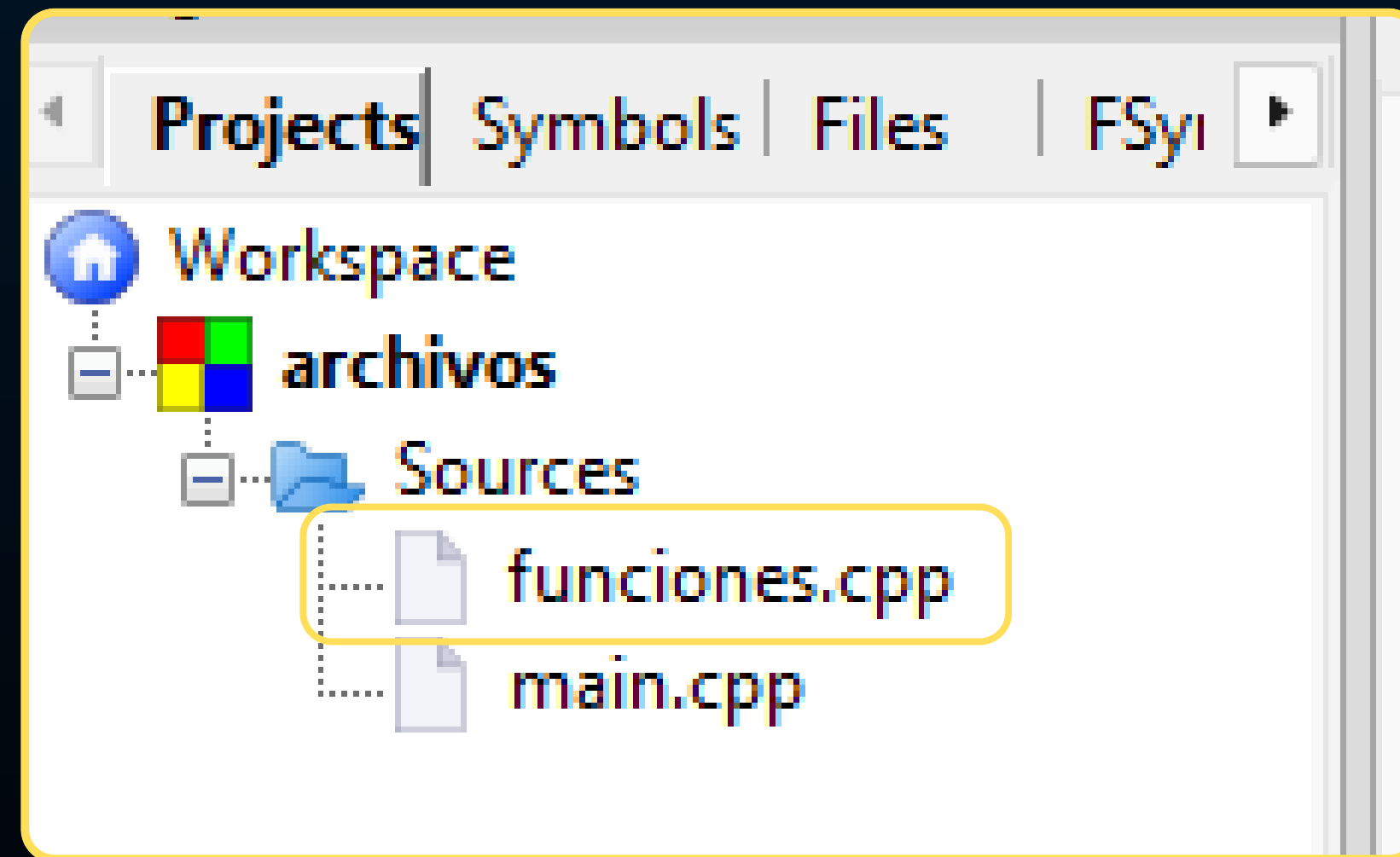
### Tildamos la opción: Add to active project.

Esto permite que el proyecto reconozca al archivo como parte del programa.

### Tildamos la opción: Debug y Release

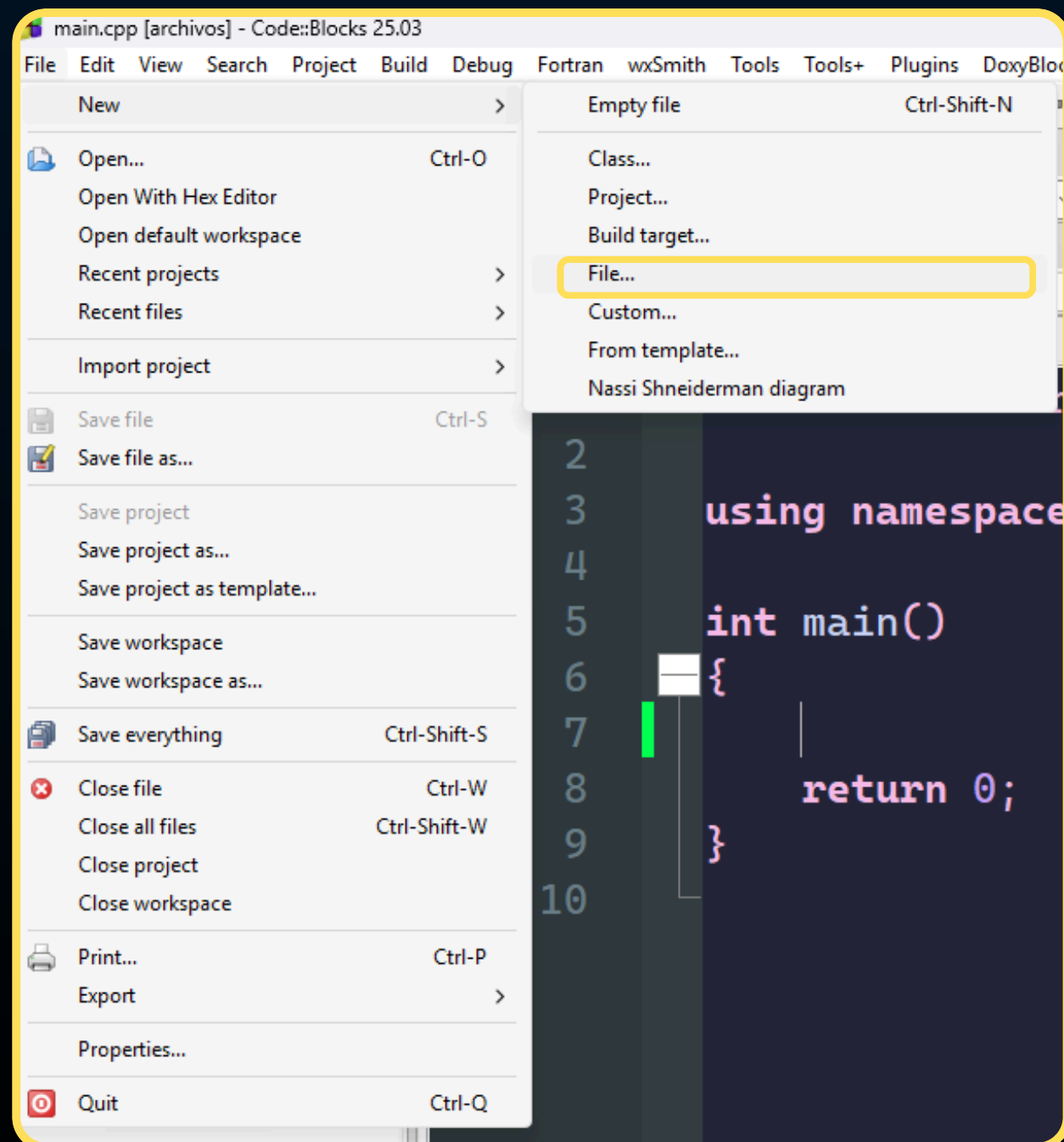
Nos permite por debugear y compilar el código

# ¿Cómo creamos archivos .cpp?

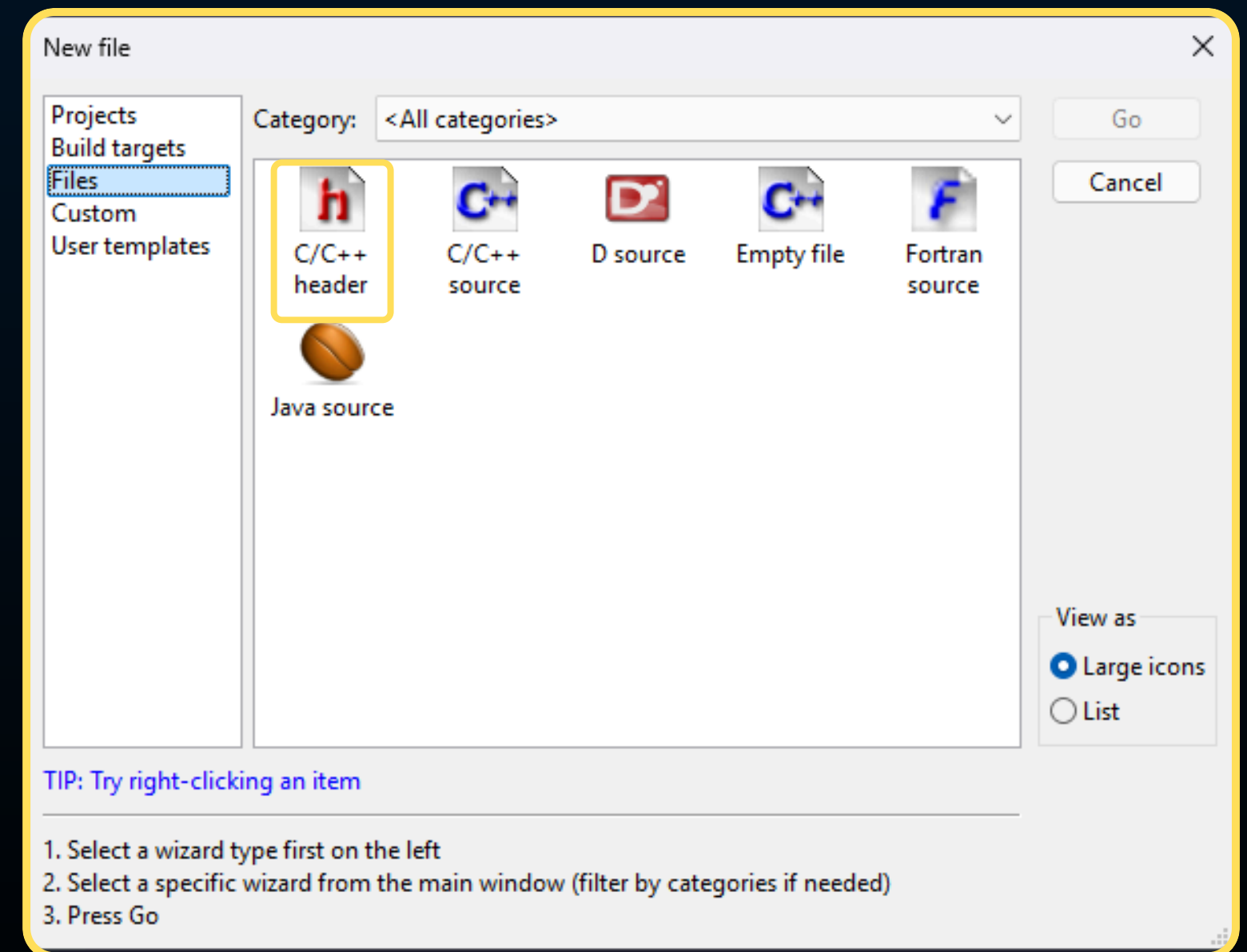


# ¿Cómo creamos archivos .h?

1 File > New > File ...



2 Files > C/C++ header



# ¿Cómo creamos archivos .h?

## 3 C/C++ source

C/C++ header

Please enter the file's location and name and whether to add it to the active project.

Filename with full path:  ...

Header guard word:

☒ Add file to active project

In build target(s):

☒ Debug  
☐ Release

All None

< Back Finish Cancel

### Directorio del archivo.

Mantenemos el mismo que el proyecto.

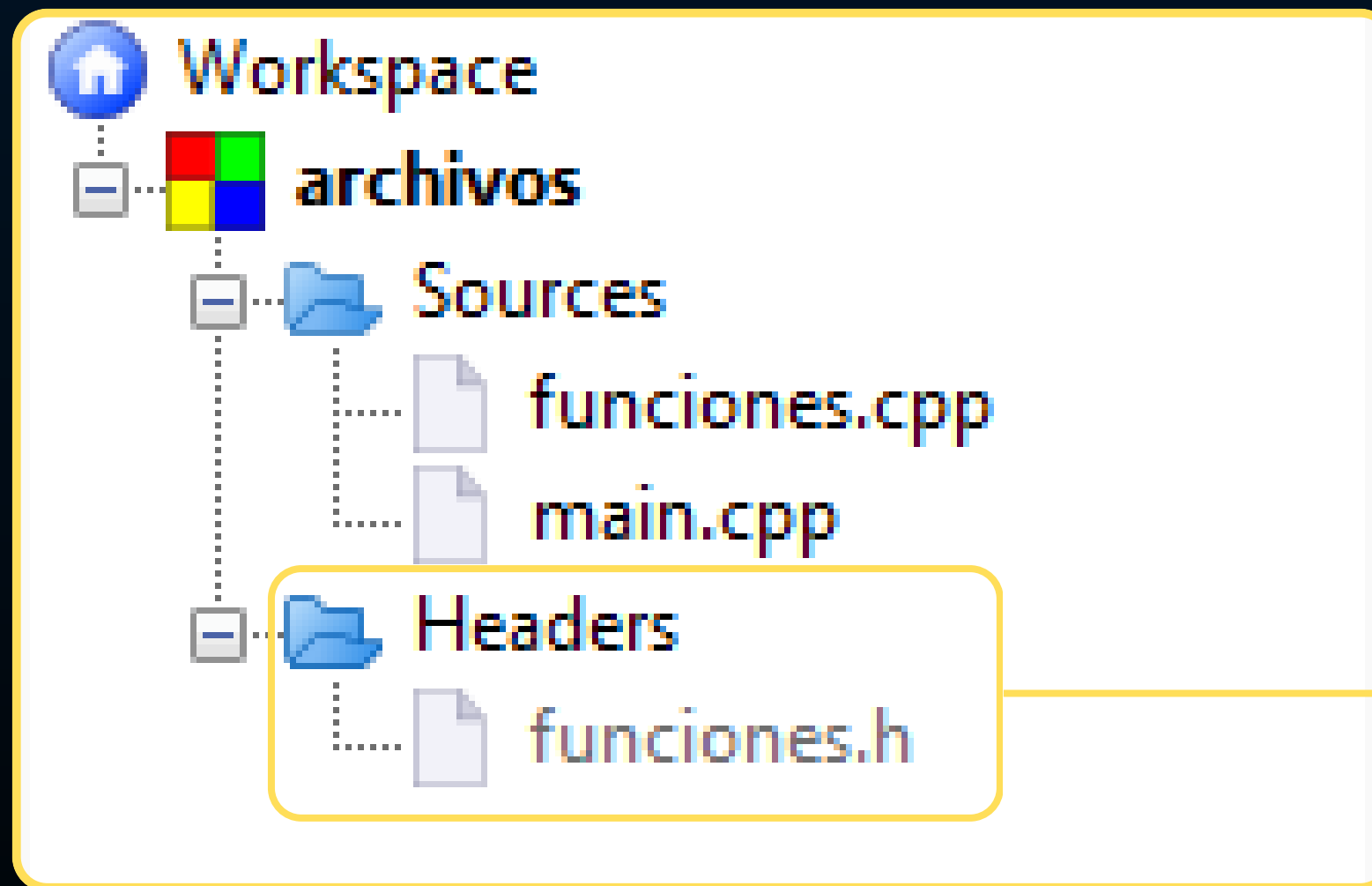
### Tildamos la opción: Add to active project.

Esto permite que el proyecto reconozca al archivo como parte del programa.

### No tildamos la opción: Debug y Release

No vamos a compilar o debugear este código.

# ¿Cómo creamos archivos .cpp?



## Headers

Se crea una nueva carpeta y coloca los archivos .h dentro.



# Archivos .h

- ◆ Solo **declaraciones** de funciones.
- ◆ Se usa una protección de inclusión (**#ifndef**, **#define**, **#endif**) para evitar errores si se incluye varias veces en otros archivos.

```
#ifndef FUNCIONES_H
#define FUNCIONES_H

// PROTOTIPO
void mostrarMensaje();

#endif
```

funciones.h

# Archivos .cpp

- ◆ Solo definiciones de funciones.
- ◆ Incluimos `#include` para que el compilador sepa que estas funciones existen.
- ◆ Incluimos a `<iostream>` para poder utilizar los métodos `cin` y `cout`

```
#include <iostream>
using namespace std;

#include "funciones.h"

void mostrarMensaje()
{
    cout << "Hola desde la función" << endl;
}
```

funciones.cpp

# Main

- ◆ `#include` copia el contenido del `.h` dentro del archivo actual.
- ◆ Al copiar el contenido del `.h` mantendría el orden que veníamos llevando antes con los prototipos por encima del `main`. Lo que permite que el compilador sepa de la existencia de las funciones.
- ◆ Cuando se compila el proyecto, todos los `.cpp` se combinan.

```
#include <iostream>
using namespace std;

#include "funciones.h"

int main()
{
    mostrarMensaje();

    return 0;
}
```

main.cpp